

Project Plan: ML-Based Computer Vision IoT Car

IoT & AI Systems Development Team

August 12, 2026

Abstract

This document outlines the project plan for designing and building an Internet of Things (IoT) vehicle controlled by computer vision and machine learning. Unlike traditional hardware-based controllers, this system utilizes a laptop webcam to capture live video feeds of the user's hand. An ML model trained on a gesture dataset classifies the hand gestures in real-time and transmits directional commands wirelessly to an ESP32-based robotic car over a local Wi-Fi network.

1 Introduction

The objective of this project is to integrate edge AI with IoT robotics. By leveraging machine learning for image processing on a laptop, the system eliminates the need for wearable sensors. The user can simply perform gestures in front of the camera, and the trained model will translate these visual cues into commands to drive the car forward, backward, left, right, or to stop.

2 System Architecture

The project relies on a Client-Server IoT architecture operating over a local Wi-Fi network:

- **Vision Controller (Laptop/PC):** Acts as the brain. A Python script uses OpenCV to capture webcam frames. An ML model (e.g., a CNN or a lightweight classifier built on top of MediaPipe Hand Landmarks) processes the frames to classify the gesture. It then sends the command via UDP/TCP sockets or an MQTT broker.
- **IoT Receiver (The Car):** Mounted on a custom chassis. An ESP32 microcontroller connects to the same Wi-Fi network, listens for incoming commands, and drives the L298N motor controller accordingly.

3 Requirements and Bill of Materials

3.1 Hardware Requirements

Since the transmitter logic is handled by the laptop, the hardware requirements are exclusively focused on the vehicle. This project is designed to be completely solder-free, provided the correct components are purchased.

3.2 Software & AI Requirements

- **AI/ML Environment (Laptop):** Python 3.x, OpenCV (for video capture), MediaPipe (optional, for extracting hand skeleton data to improve training efficiency), TensorFlow/Keras or Scikit-Learn (for model training and inference).

Subsystem	Component	Quantity
Controller	Laptop/PC with Webcam	1
Receiver (Car)	ESP32 Microcontroller (Wi-Fi enabled)	1
	L298N Motor Driver Module	1
	DC Gear Motors (Must be pre-wired/have wires attached)	4
	Wheels	4
	Custom/Acrylic Robot Chassis	1
	18650 Li-ion Batteries (with holder)	2
	Jumper Wires (Female-to-Female <i>and</i> Male-to-Female)	1 Set

Table 1: Hardware Bill of Materials

- **Car Environment:** Arduino IDE or PlatformIO.
- **Communication Protocol:** Wi-Fi (UDP/TCP Sockets) or MQTT.

4 Hardware Assembly Guide

This section details the physical construction and electrical wiring of the IoT car receiver unit based on the components listed in Table 1.

4.1 Mechanical Assembly

1. **Motor Mounting:** Attach the four DC gear motors to the custom/acrylic robot chassis using the provided brackets and screws. Ensure the motor shafts are aligned correctly.
2. **Wheel Attachment:** Press-fit the four wheels onto the exposed shafts of the DC gear motors.
3. **Component Placement:** Secure the L298N motor driver module, the ESP32 microcontroller, and the 18650 battery holder onto the top deck of the chassis. Use standoffs or double-sided tape as appropriate to prevent short circuits on the chassis.

4.2 Electrical Wiring

Follow these steps to connect the electronic components. Note: Ensure the batteries are removed or the power switch is OFF during wiring. No soldering is required for this setup.

1. Motor Wiring (Parallel Configuration via Screw Terminals):

- *Note: The L298N uses screw terminals for power and motors. Strip a small amount of insulation from the motor wires, insert them into the terminal block, and tighten the screw.*
- Connect the positive (+) wires of the two **Left-side motors** together, and connect them to the OUT1 terminal of the L298N.
- Connect the negative (-) wires of the two **Left-side motors** together, and connect them to the OUT2 terminal of the L298N.
- Connect the positive (+) wires of the two **Right-side motors** together, and connect them to the OUT3 terminal of the L298N.
- Connect the negative (-) wires of the two **Right-side motors** together, and connect them to the OUT4 terminal of the L298N.

2. Power Distribution (Common Ground is Critical):

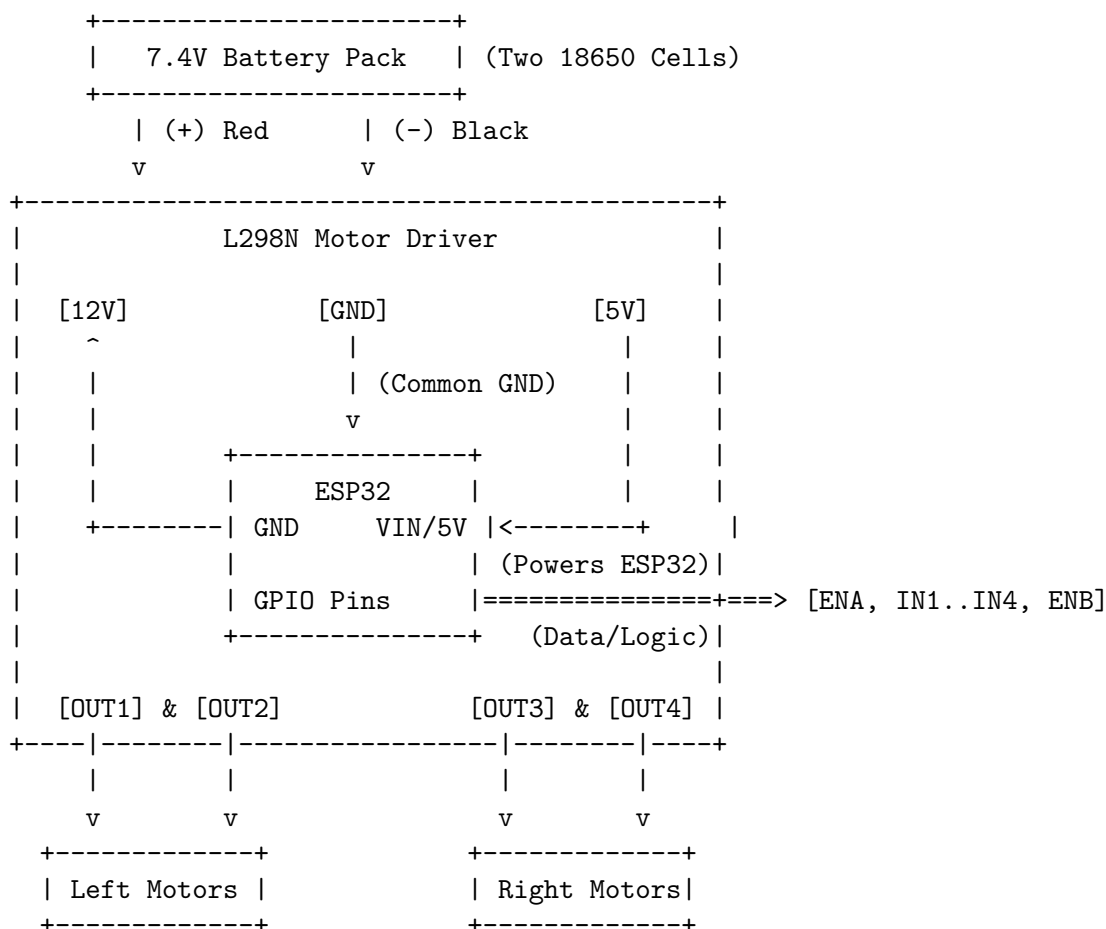
- Connect the positive terminal (Red wire, approx 7.4V from two 18650s) of the battery holder to the 12V (V_{motor}) input screw terminal of the L298N.
- Connect the negative terminal (Black wire) of the battery holder to the GND screw terminal of the L298N.
- **CRITICAL:** Run a jumper wire from the GND screw terminal of the L298N to a GND pin on the ESP32 to establish a common ground for logic signals.
- Powering the ESP32: If your L298N has the 5V regulator jumper in place (enabled), run a wire from the L298N's 5V screw terminal to the VIN (or 5V) pin on the ESP32.

3. Control Logic Wiring (ESP32 to L298N via Jumper Wires):

- *Note: The L298N logic inputs are male header pins. You will need Female-to-Female jumper wires to connect these to the ESP32 (which also typically has male pins).*
- Remove the default jumpers on ENA and ENB if you want PWM speed control.
- Wire the ESP32 GPIO pins to the L298N logic pins according to the schematic table below.

4.3 Wiring Diagram and Pinout Schematic

Because external schematic image files cannot be directly embedded in this document, the following text-based block diagram illustrates the overall power and data flow between the main components. This provides a visual structural map to accompany the detailed pin mapping.



The following table provides the exact pin-to-pin mapping required to establish the connections between the ESP32, the L298N, the battery pack, and the motors.

From Component / Pin	To Component / Pin
Power Supply (7.4V Battery)	
Battery Positive (+)	L298N 12V Terminal
Battery Negative (-)	L298N GND Terminal
ESP32 Power & Ground	
ESP32 VIN (or 5V input)	L298N 5V Terminal (if 5V jumper is ON)
ESP32 GND	L298N GND Terminal (Common Ground)
ESP32 Logic to L298N	
ESP32 GPIO 14	L298N ENA (Motor A Speed/Enable)
ESP32 GPIO 27	L298N IN1 (Motor A Direction 1)
ESP32 GPIO 26	L298N IN2 (Motor A Direction 2)
ESP32 GPIO 25	L298N IN3 (Motor B Direction 1)
ESP32 GPIO 33	L298N IN4 (Motor B Direction 2)
ESP32 GPIO 32	L298N ENB (Motor B Speed/Enable)
L298N to Motors	
L298N OUT1 & OUT2	Front-Left and Rear-Left Motors (Parallel)
L298N OUT3 & OUT4	Front-Right and Rear-Right Motors (Parallel)

Table 2: Detailed Wiring Schematic for ESP32 and L298N

5 Gesture Mapping & Model Classes

The ML model will be trained on a custom dataset mapping specific visual hand signs to discrete classifications. The output string or integer will be sent to the car.

Visual Hand Gesture	ML Class Output	Car Movement
Open Palm facing camera	"FORWARD"	Move Forward
Closed Fist	"STOP"	Stop Motors
Thumb pointing Left	"LEFT"	Turn Left
Thumb pointing Right	"RIGHT"	Turn Right
Peace Sign (Two fingers)	"BACKWARD"	Move Backward

Table 3: Proposed ML Classification to Movement Mapping

6 Implementation Timeline

The project will be executed in four primary phases:

Phase 1: Data Collection and Car Assembly (Week 1)

- Assemble the 4-wheel-drive chassis, motor driver, and ESP32.
- Write a script to collect a video/image dataset using the laptop webcam, capturing hundreds of variations for each of the 5 gesture classes.
- Annotate/organize the dataset for training.

Phase 2: ML Model Training and Validation (Week 2)

- Develop the Machine Learning pipeline. (*Recommendation: Extract hand landmarks using MediaPipe, then train a lightweight Neural Network or Random Forest on the landmark coordinates. This is faster and more robust than raw image CNNs*).
- Train the model on the collected dataset.
- Test the model locally via webcam to ensure high real-time accuracy and low latency.

Phase 3: IoT Communication Setup (Week 3)

- Write the ESP32 firmware to connect to the local Wi-Fi network and host a simple UDP server or connect to an MQTT broker.
- Write the motor control logic (translating incoming strings like "FORWARD" into GPIO HIGH/LOW signals).
- Update the Python ML script on the laptop to transmit the classified gesture string over the network to the ESP32's IP address.

Phase 4: End-to-End Integration and Testing (Week 4)

- Run the full system: Start the Python ML script and power on the IoT Car.
- Test responsiveness. Measure network latency and model inference time.
- Fine-tune the motor speeds (using PWM) and adjust the ML model confidence thresholds to prevent accidental movements.

7 Conclusion

By transitioning to a Machine Learning and Computer Vision approach, this project demonstrates a highly advanced, contactless interface for IoT control. The laptop handles the heavy computational load of real-time AI inference, while the ESP32 efficiently handles edge execution. Future upgrades could include deploying the ML model directly onto an edge-AI board (like a Raspberry Pi or Jetson Nano) mounted directly on the car for a fully autonomous, self-contained robotic system.